Part 1 — Pure GEMM: make these shapes fast (float and int8)

This part is just GEMM. No language model, no story, nothing to install. You are given an interface and a set of matrix shapes; your job is to implement matrix multiply behind that interface and make it **correct first, then fast** — in **float32** and in **int8**. Everything you need to develop and test runs offline on one CPU core.

> **The glimpse (why you're doing this).** These particular shapes are the matrix multiplies inside a small language model that writes children's stories. In **Part 2** you will drop this exact GEMM into that model and watch it generate text faster; in **Part 3** you will do it in int8. But you do **not** need any of that now. Nail the GEMM here, in isolation — the model is the payoff, not the prerequisite.

1.1 The interface — one function

You implement a single function (`src/student_gemm.h`):

```
// C[M*N] = A[M*K] · B[K*N]   — all row-major, float32, contiguous
void student_gemm(const float *A, const float *B, float *C, int M, int K, int N);
```

Standard GEMM: `C = A·B` . Row-major means element `(r,c)` of an `R×Cols` matrix is at `r*Cols + c` . The dimensions arrive **at runtime** — your one implementation must handle every shape below (and every `N`), not a single hard-coded size.

- **M** — rows of `A` and `C` (the output dimension)
- **K** — the shared/reduction dimension (cols of `A` , rows of `B`)

- **N** — cols of `B` and `C` (**how many vectors** you multiply at once)

A reference implementation ships in `src/student_gemm_ref.c` so the tester runs on day one. Replace it with your optimized version — same signature, nothing else changes.

1.2 The target workload

Your kernel must handle these four `(M, K)` shapes — every dimension is a multiple of 32, which you can lean on for tiling:

Name	M (out)	K (in)	Notes
attn-proj	288	288	square
ffn-up	768	288	tall (widen)
ffn-down	288	768	wide (shrink)
classifier	32000	288	very tall — the FLOP monster

And two regimes of `N`:

- `N = 1` — a single vector. This is a matrix-**vector** product (a *GEMV*). It is **memory-bound**: you read the entire `M×K` matrix to produce only `M` numbers, so every weight is used exactly once. There is little to exploit; you are limited by how fast you can stream the matrix.
- `N > 1` — several vectors stacked as columns. Now it is a true matrix-**matrix** product (a *GEMM*), and each element of `A` is reused across all `N` columns. This is where tiling, blocking, and on-chip reuse pay off, and where a good implementation pulls far ahead of the reference. **Optimizing the `N > 1` case is the heart of this part.**

Why both? Because in Part 2 the model uses `N = 1` when it generates one token at a time, and `N > 1` when it processes many at once — and you will discover that the `N` you feed the kernel decides everything about throughput. Design for both now.

1.3 Test it — offline, against a golden

`scripts/synth_cases.sh` generates random test matrices of the four shapes (with a double-precision golden), and `scripts/test_gemm.sh` runs your `student_gemm` on them, checks correctness (`1e-2` tolerance), and reports GFLOP/s:

```
cd mini1-gemm
./scripts/synth_cases.sh                # makes cases/synth_*  (N = 1 and 32 by default)
./scripts/test_gemm.sh                  # tests the reference impl
GEMM_SRC=path/to/your_gemm.c ./scripts/test_gemm.sh    # tests YOURS
```

The tester prints PASS/FAIL and a GFLOP/s figure per shape — the format is:

```
=== testing student_gemm against 4 GEMM case(s) ===
  synth_288_to_288_N1.bin    M=288    K=288   N=1   PASS   … GFLOP/s
  synth_288_to_32000_N1.bin  M=32000  K=288   N=1   PASS   … GFLOP/s
  synth_288_to_768_N1.bin    M=768    K=288   N=1   PASS   … GFLOP/s
  synth_768_to_288_N1.bin    M=288    K=768   N=1   PASS   … GFLOP/s
=== ALL PASS (4/4 passed) ===
```

The reference `student_gemm` is a naive, single-threaded triple loop — **run it once and record its GFLOP/s as your floor**, then beat it. Re-run with `NS="1 8 32 64" ./scripts/synth_cases.sh` to see how your GFLOP/s changes as `N` grows: a good implementation climbs steeply with `N` , a naive one stays flat.

1.4 The int8 half

Real accelerators don't do this in float; they do it in **int8** — 4× less data to move and far more multiply-adds per unit of hardware. Part 1 asks you to build the integer version too (`src/student_gemm_int8.h`):

```
// C[M*N] fp32 = grouped-int8 product of Aq[M*K] and Bq[K*N]
void student_gemm_int8(const int8_t *Aq, const float *As,   // values + per-group fp32 sc
```

```
                    const int8_t *Bq, const float *Bs,
                    float *C, int M, int K, int N, int GS);
```

Group quantization (Q8_0): the reduction dimension `K` is cut into groups of `GS` elements; each group stores int8 values plus one fp32 **scale** per operand. You accumulate the inner product in **int32** within a group (this is the integer speed you want), then convert back to fp32 with the two scales at the group boundary:

```
C[i,j] = Σ_groups g ( As[i,g] · Bs[g,j] · Σ_{k in group g} (int32) Aq[i,k]·Bq[k,j] )
```

Why groups instead of one scale for the whole matrix? A single scale would have to cover the largest magnitude in thousands of numbers, crushing the small ones to zero. Per-group scales keep precision local — the standard trick in real int8 inference.

Test it the same way (self-contained, built-in golden, four shapes):

```
./scripts/test_gemm_int8.sh              # reference int8 impl, N=1, GS=32
N=32 ./scripts/test_gemm_int8.sh         # batched
GEMM_INT8_SRC=path/to/your_int8.c ./scripts/test_gemm_int8.sh
```

The int8 tester prints PASS/FAIL and GOP/s per shape — the format is:

```
=== int8 GEMM test (N=1, GS=32) ===
  attn-proj    M=288    K=288  N=1  GS=32  PASS  … GOP/s
  ffn-up       M=768    K=288  N=1  GS=32  PASS  … GOP/s
  ffn-down     M=288    K=768  N=1  GS=32  PASS  … GOP/s
  classifier   M=32000  K=288  N=1  GS=32  PASS  … GOP/s
=== ALL PASS ===
```

(`GS` must divide `K` ; 32 divides both 288 and 768.)

1.5 What Part 1 asks you to deliver

1. A `student_gemm` (float) that passes all four shapes and is **substantially faster than the reference**, especially as `N` grows. Report your GFLOP/s per shape at a couple of `N` values.

2. A `student_gemm_int8` that passes all four shapes. Report your GOP/s.
3. A short note on **what made it fast** — how you tiled/blocked, where the reuse came from, why `N > 1` helped, and what limited you at `N = 1`.

That is the whole mini-project core, and it stands on its own. When you're ready to see what it powers — a real model writing stories, and your kernel making it write them faster — turn to **Part 2**.

Checkpoint

- [ ] `test_gemm.sh` with your float kernel: all shapes PASS, GFLOP/s well above the naive reference floor you recorded, and rising with `N`.
- [ ] `test_gemm_int8.sh` with your int8 kernel: all shapes PASS.
- [ ] Your "what made it fast" note (tiling, reuse, the `N=1` vs `N>1` gap).